Объект хандалтат программчлалын удамшил

(Лабораторийн ажил 6)

МУИС, МТЭС, МКУТ-ийн суралцагч

А. Баяржавхлан, 24B1NUM0876@stud.num.edu.mn

Э. Баттамир, 24B1NUM0616@stud.num.edu.mn

Б. Маргад-Эрдэнэ, 24B1NUM1418@stud.num.edu.mn

6.1 Оршил

Объект хандалтат програмчлалын үндсэн ойлголтуудын нэг нь удамшил бөгөөд энэ нь нэг классын шинж чанар, үйлдлийг өөр класс дахин ашиглах боломжийг бүрдүүлдэг. Удамшлын механизмыг ашигласнаар програмын кодыг дахин ашиглах, бүтэц зохион байгуулалтыг сайжруулах, илүү ойлгомжтой систем байгуулах боломжтой болдог. C++ хэл дээр удамшлыг ашиглан эцэг болон хүүхэд классуудын хооронд хамаарал үүсгэж, нийтлэг шинжүүдийг нэг дор төвлөрүүлэн тодорхойлж болно. Энэхүү даалгаврын хүрээнд объект хандалтат програмчлалын удамшлын ойлголтыг онолын болон практик талаас нь судална. Тухайлбал, удамшлын тодорхойлолт, удамшлын горимууд болох public, private, protected хандалтын түвшнүүд, удамшлын давуу талууд болон төрөлүүдийг тайлбарлана. Мөн Shape, 2DShape, Circle, Square, Triangle классуудын удамшлын бүтэц зохион байгуулалтыг C++ хэл дээр хэрэгжүүлж, геометрийн дүрсүүдийн талбай болон периметрийг тооцоолох програм боловсруулна.

6.2 Зорилго

Энэхүү лабораторийн ажлын үндсэн зорилго нь Объект хандлагат програмчлалын тулгуур ойлголт болох Удамшил (Inheritance)-ийн мөн чанарыг танин мэдэж, C++ хэл дээрх бүтэц бүхий классуудыг зохиомжлох туршлага хуримтлуулахад оршино. Үүний хүрээнд дараах дэд зорилгуудыг хэрэгжүүлнэ:

1. Ерөнхийллийн арга: Классуудын нийтлэг шинж чанарыг эх классад нэгтгэж, кодын давхцлыг арилгах замаар зохиомжийн оновчтой шийдэл гаргах.
2. Хандалтын удирдлага: public, protected, private горимуудын ялгааг таньж, өгөгдлийн нууцлал болон удамших боломжийг практикт хэрэгжүүлэх.
3. Кодын дахин ашиглалт: Бэлэн байгаа классыг өргөтгөн шинэ функцүүд нэмэх замаар програмчлалын бүтээмжийг нэмэгдүүлэх.
4. Багаар ажиллах ур чадвар: Төслийн даалгаврыг багийн гишүүдийн дунд оновчтой хуваарилж, кодын болон тайлангийн нэгдмэл байдлыг хангах.

6.3 Онолын судалгаа

1. **Удамшил гэж юу вэ?** (тодорхойлолт, C++ хэл дээр хэрхэн хэрэгжүүлдэг талаар бичиж, жишээ кодоор тайлбарлана)

Удамшил гэдэг нь Объект хандалтат программчлалын суурь ойлголт бөгөөд нэг класс өөрийн шинж чанар болон функцуудыг өөр нэг классад өвлүүлэн ашиглуулах технологийг хэлнэ[1]. C++ хэл дээр удамшлыг хэрэгжүүлэхдээ хүүхнэ классыг зарлахдаа : операторыг ашиглаж эцэг классыг дууддаг. Энэ нь кодыг дахин ашиглах болон програмын логик бүтцийг шатлалтай болгох ач холбогдолтой[1].

```
class Square : public Shape2D {
   private:
         double x0, y0;
         double x1, y1;
         double x2, y2;
         double x3, y3;
         double side;
   public:
         double area() override;
         double perimeter() override;
         void printName();
};
```

2. **Удамшлын горим.** public, private, protected горимын талаар тайлбарлаж жишээгээр батална.

Удамшлын горим нь эх классын гишүүд хүүхэд класс руу шилжихдээ ямар түвшний хандалттай болохыг тодорхойлдог. public горимоор удамшихад эх классын нээлттэй гишүүд хүүхэд класс дотор нээлттэй хэвээр үлдэнэ[1]. protected горим нь тухайн өгөгдлийг зөвхөн удамшсан классууд дотор хандах боломжтой болгож, гадны нөлөөнөөс хамгаалдаг[2]. Харин private горим нь бүх өгөгдлийг тухайн класс дотор нь түгжиж, цааш удамших боломжийг хязгаарладаг[1].

```
class Shape {
   protected:
         string name;
   public:
         void setName(const string& name);
         string getName() const;
};
```

Хүүхэд класст:

```
void Square::printName() {
   cout << "Shape name: " << name << endl;
}
```

3. **Удамшлыг яагаад хэрэглэх ёстой вэ**, давуу талуудыг тоочин бичиж бодит жишээн дээр тайлбарла.

Удамшлыг хэрэглэх гол шалтгаан нь кодыг дахин ашиглах замаар цаг хугацаа хэмнэх, алдааг багасгах явдал юм. Эх класс дээр нэг удаа бичсэн логикийг олон удам классууд шууд өвлөн авснаар програм илүү цэгцтэй болдог[2].

Square калсст name гэх гишүүн өгөгдөл байхгүй бөгөөд түүний эцэг класс болох Shape-ийн гишүүн өгөгдөлд хандаж байна. Өөрөөр хэлбэл бодит амьдралд ерөнхий шинж чанарыг нэг удаа тодорхойлж хүүхэд классуудад өвлүүлэх байдлаар кодын дахин ашиглалтыг нэмэгдүүлж байна.

```
void Square::printName() {
   cout << "Shape name: " << name << endl;
}
```

4. **Удамшлын хэдэн төрөл байдаг вэ?** Тус бүрийг тайлбарлан, жишээ код бичнэ.

C++ хэлэнд удамшлыг бүтцийн хувьд дараах 4 үндсэн төрөлд хуваажүздэг. Үүнд:

Дан удамшил (Single Inheritance): Нэг эх классаас зөвхөн нэг удам класс үүсэхийг хэлнэ. Энэ нь удамшлын хамгийн энгийн бөгөөд нийтлэг хэлбэр юм[3].

```
class Shape {
   protected:
         string name;
   public:
         void setName(const string& name);
         string getName() const;
};

class Shape2D : public Shape {
   public:
         virtual double area() = 0;
         virtual double perimeter() = 0;
```

```
};

Shape -> Shape2d
```

Олон түвшний удамшил (Multilevel Inheritance): Удамшсан классаас дахин өөр класс салбарлан удамшихыг хэлнэ. Энэ тохиолдолд хамгийн сүүлийн удам класс нь өвөг классынхаа бүх шинж чанарыг өвлөн авдаг[3].

```
class Shape {
   protected:
         string name;
   public:
         void setName(const string& name);
         string getName() const;
};

class Shape2D : public Shape {
   public:
         virtual double area() = 0;
         virtual double perimeter() = 0;
};

class Circle : public Shape2D {
   private:
         double crd_1;
         double crd_2;
         double radius;
   public:
         double area() override;
         double perimeter() override;

         void printName();
};

Shape -> Shape2d -> Circle
```

Шаталсан удамшил (Hierarchical Inheritance): Нэг эх классаас хэд хэдэн салбар удам классууд зэрэг удамшихыг хэлнэ. Энэ нь нэг ерөнхий ойлголтыг хэд хэдэн ангилалд хуваахад ашиглагддаг[3].

```
class Square : public Shape2D
class Circle : public Shape2D
class Triangle : public Shape2D
```

```
Shape2d -> Square
        -> Circle
        -> Triangle
```

Олон удамшил (Multiple Inheritance): Нэг удам класс нь хоёр буюу түүнээс дээш эх классаас зэрэг шинж чанар өвлөн авахыг хэлнэ. Энэ нь C++ хэлний нэг онцлог давуу тал[3].

```
class Father { };
class Mother { };

class Child : public Father, public Mother { };

Father ->
       -> Child
Mother ->
```

6.4 Хэрэгжүүлэлт

Бодлого: Shape классаас 2DShape гэдэг класс удамшина. 2DShape классаас Circle, Square, Triangle классууд удамшина. Доор хүүхэд классуудын шинж болон функцууд өгөгдсөн бол Shape, 2DShape классуудын шинжийг ерөнхийллийн аргаар зохиомжилж удамшуул.

```
class Shape {
   protected:
         string name;
   public:
         void setName(const string& name);
         string getName() const;
};

class Shape2D : public Shape {
   public:
         virtual double area() = 0;
         virtual double perimeter() = 0;
};
```

1. Тойрог: гишүүн өгөгдөл-тойргийн төвийн координат, радиус, нэр; гишүүн функц-талбай олох, тойргийн урт тооцох

```
double Circle::setter(const double n, const double cx) {

   radius = n;

   crd_1 = cx;
   crd_2 = cx;

   return radius;
}

void Circle::getter() {
   cout << "Radius: " << radius << endl;
}

double Circle::area() {
   return M_PI * radius * radius;
}

double Circle::perimeter() {
   return 2 * M_PI * radius;
}
```

2. Квадрат: гишүүн өгөгдөл-оройнуудын координат, талын урт, нэр; гишүүн функц-талбай олох, приметр тооцох.

```
double Square::setter(const double n, const double cx, const double
cy) {

   side = n;

   x0 = cx;
   y0 = cy;

   x1 = cx + side;
   y1 = cy;

   x2 = cx + side;
   y2 = cy - side;

   x3 = cx;
   y3 = cy - side;

   return side;
}

void Square::getter() {
   cout << "Side: " << side << endl;
}

double Square::area() {
   return side * side;
}

double Square::perimeter() {
   return 4 * side;
}
```

3. Зөв гурвалжин: гишүүн өгөгдөл-оройнуудын координат, талуудын урт, нэр; гишүүн функц-талбай, приметр тооцох.

```
double Triangle::setter(const double n, const double cx, const
double cy) {

   side = n;

   x0 = cx;
```

```
    y0 = cy;

    x1 = cx - side/2;
    y1 = cy - (sqrt(3)/2)*side;

    x2 = cx + side/2;
    y2 = cy - (sqrt(3)/2)*side;

    return side;
}

void Triangle::getter() {
    cout << "Side: " << side << endl;
}

double Triangle::area() {
    return (sqrt(3)/4) * side * side;
}

double Triangle::perimeter() {
    return 3 * side;
}
```

6.5 Багийн ажиллагаа

1. Гишүүдийн оролцоо

Э. Баттамир - square.cpp
А. Баяржавхлан - circle.cpp
Б. Маргад-Эрдэнэ - triangle.cpp

2. Тулгарсан асуудал

1) Функцын нэрийг өөрөөр нэрлэж байсан учир header файлд нэгтгэж тодорхойлсон.
2) Кодын формат зөрж байсан тул гүйцэтгэлийн дараа бүгдийг нэг стандарттай болгосон.
3) Github-д push хийхэд нэг хавтаст олон commit үүсэж төвөгтэй байдал үүссэн тул тус бүрт хавтас үүсгэж main.cpp хавтаст дуудсан.

3. Сурсан мэдлэг

Багийн гишүүдийн гүйцэтгэх ажлыг тэнцүү хуваах, даалгаврыг илүү хурдан бөгөөд үр дүнтэй гүйцэтгэх боломжтойг ойлгосон. Мөн багийн гишүүд хоорондоо кодоо нэг стандартын дагуу бичих, функц болон хувьсагчийн нэршлийг ижил байдлаар ашиглах нь кодыг нэгтгэхэд хялбар болгодогийг сурсан.

Үүнээс гадна C++ хэл дээр класс, удамшил, virtual функц, header файл ашиглан кодыг олон файлд зохион байгуулах арга барилыг практик дээр хэрэгжүүлж сурсан. Github ашиглан хамтран ажиллах явцад commit болон push хийх дараалыг ойлгож, багийн гишүүдийн кодыг нэг төсөлд нэгтгэх туршлага хуримтлуулсан.

6.6 Дүгнэлт

Энэхүү лабораторийн ажлаар удамшил ашиглан Shape болон Shape2d классуудыг зохиомжилж, түүнээс Circle, Square, Triangle классуудыг салбарлуулан үүсгэв. Ингэснээр дүрс бүрт давтагддаг нэр, координат зэрэг өгөгдлүүдийг эх классаас нь нэг удаа дамжуулж, кодын бичиглэлийг цэгцэлсэн. Мөн protected горимыг ашиглан хүүхэд классууд эх классын өгөгдөлд шууд хандах боломжийг бүрдүүлж, дүрс бүрийн талбай, периметрийг бодох функцүүдийг тус тусад нь хэрэгжүүллээ. Багаар ажиллах явцад ажил үүргээ хуваарилж, кодоо нэгтгэн даалгаврын дагуу даалгаврын бүтцийг бүрэн угсарч гүйцэтгэв.

6.7 Ашигласан материал

1. М. Золжаргал багшийн лекцийн материал
2. Объект хандлагат технологийн C++ программчал, Ж. Пүрэв, 2008, Улаанбаатар
3. geeksforgeeks

6.8 Хавсралт

1. oop.h

```
#ifndef OOP_H
#define OOP_H

#include<string>
#include<iostream>
#include<cmath>
using namespace std;

class Shape {
   protected:
         string name;
   public:
         void setName(const string& name);
         string getName() const;
};

class Shape2D : public Shape {
   public:
         virtual double area() = 0;
         virtual double perimeter() = 0;
};

class Circle : public Shape2D {
   private:
         double crd_1;
         double crd_2;
         double radius;
   public:
         double area() override;
         double perimeter() override;

         void printName();
};

class Square : public Shape2D {
   private:
         double x0, y0;
         double x1, y1;
         double x2, y2;
         double x3, y3;
         double side;
   public:
         double area() override;
         double perimeter() override;
```

```
        void printName();
};

class Triangle : public Shape2D {
   private:
        double x0, y0;
        double x1, y1;
        double x2, y2;
   public:
        double area() override;
        double perimeter() override;
        void printName();
};

#endif
```

2. circle.cpp

```
#include "../oop.h"

double Circle::setter(const double n, const double cx) {

   radius = n;

   crd_1 = cx;
   crd_2 = cx;

   return radius;
}

void Circle::getter() {
   cout << "Radius: " << radius << endl;
}

double Circle::area() {
   return M_PI * radius * radius;
}

double Circle::perimeter() {
   return 2 * M_PI * radius;
}

void Circle::printName() {
   cout << "Shape name: " << name << endl;
}

void Circle::showCoordinate() {
```

```
    cout << "Center: (" << crd_1 << ", " << crd_2 << ")" << endl;
}
```

3. shape.cpp

```
#include "../oop.h"

void Shape::setName(const string& name) {
    this->name = name;
}

string Shape::getName() const {
    return name;
}
```

4. Square.cpp

```
#include "../oop.h"

double Square::setter(const double n, const double cx, const double
cy) {

    side = n;

    x0 = cx;
    y0 = cy;

    x1 = cx + side;
    y1 = cy;

    x2 = cx + side;
    y2 = cy - side;

    x3 = cx;
    y3 = cy - side;

    return side;
}

void Square::getter() {
    cout << "Side: " << side << endl;
}

double Square::area() {
    return side * side;
}

double Square::perimeter() {
```

```
    return 4 * side;
}

void Square::printName() {
    cout << "Shape name: " << name << endl;
}

void Square::ShowCoordinates() {

    cout << "Coordinates:" << endl;

    cout << "A(" << x0 << ", " << y0 << ")" << endl;
    cout << "B(" << x1 << ", " << y1 << ")" << endl;
    cout << "C(" << x2 << ", " << y2 << ")" << endl;
    cout << "D(" << x3 << ", " << y3 << ")" << endl;

}
```

5. triangle.cpp

```
#include "../oop.h"

double Triangle::setter(const double n, const double cx, const
double cy) {

    side = n;

    x0 = cx;
    y0 = cy;

    x1 = cx - side/2;
    y1 = cy - (sqrt(3)/2)*side;

    x2 = cx + side/2;
    y2 = cy - (sqrt(3)/2)*side;

    return side;
}

void Triangle::getter() {
    cout << "Side: " << side << endl;
}

double Triangle::area() {
    return (sqrt(3)/4) * side * side;
}
```

```
double Triangle::perimeter() {
   return 3 * side;
}

void Triangle::printName() {
   cout << "Shape name: " << name << endl;
}

void Triangle::showCoordinates() {

   cout << "Coordinates:" << endl;

   cout << "A(" << x0 << ", " << y0 << ")" << endl;
   cout << "B(" << x1 << ", " << y1 << ")" << endl;
   cout << "C(" << x2 << ", " << y2 << ")" << endl;
}
```

6. main.cpp

```
#include "oop.h"

int main () {
   Circle c;
   Square s;
   Triangle t;
   int which;

   while (true) {
         cout << "1. Circle, 2. Square, 3. Triangle, 4. Exit" <<
         endl;
         cin >> which;
         if (which == 4) break;

         double r, x, y, side;

         switch (which)
         {
               case 1: // Circle

                     cout << "Radius: ";
                     cin >> r;

                     cout << "Center coordinate (x y): ";
                     cin >> x >> y;

                     c.setName("Circle");
                     c.setter(r, x);
```

```
            c.printName();
            c.showCoordinate();

            cout << "Area: " << c.area() << endl;
            cout << "Perimeter: " << c.perimeter() << endl;
            break;
        case 2: // Square

            cout << "Side: ";
            cin >> side;

            cout << "Top-left coordinate (x y): ";
            cin >> x >> y;

            s.setName("Square");
            s.setter(side, x, y);

            s.printName();
            s.ShowCoordinates();

            cout << "Area: " << s.area() << endl;
            cout << "Perimeter: " << s.perimeter() << endl;
            break;

        case 3: // Triangle

            cout << "Side length: ";
            cin >> side;

            cout << "Top vertex coordinate (x y): ";
            cin >> x >> y;

            t.setName("Triangle");

            t.setter(side, x, y);

            t.printName();

            t.showCoordinates();

            cout << "Area: " << t.area() << endl;
            cout << "Perimeter: " << t.perimeter() << endl;
            break;

        default:
            break;
```

```
            }
        }
        return 0;
    }
```

7. MakeFile

```
all: go

circle.o: cpp/circle.cpp oop.h
      g++ -c cpp/circle.cpp -o circle.o

shape.o: cpp/shape.cpp oop.h
      g++ -c cpp/shape.cpp -o shape.o

shape2d.o: cpp/shape2d.cpp oop.h
      g++ -c cpp/shape2d.cpp -o shape2d.o

square.o: cpp/square.cpp oop.h
      g++ -c cpp/square.cpp -o square.o

triangle.o: cpp/triangle.cpp oop.h
      g++ -c cpp/triangle.cpp -o triangle.o

main.o: main.cpp oop.h
      g++ -c main.cpp -o main.o

go: circle.o shape.o shape2d.o square.o triangle.o main.o
      g++ circle.o shape.o shape2d.o square.o triangle.o main.o -o
go

clean:
rm -v *.o go
```

Даалгавар байршиж буй холбоос: Github